HROne Technical Assignment - Engineering Implementation & Compliance Report

Role: Software Engineer Trainee | **Stack:** Python, FastAPI, MongoDB | **Date:** October 2026

1. Executive Summary & Deliverables Verification

This document summarizes all engineering modifications, defect corrections, business rule implementations, and architectural decisions completed for the Employee Attendance & Analytics API assignment. The application adheres strictly to the OpenAPI 3.0.3 specification, DATA_MODEL requirements, and business rules R1 through R10.

Deliverable	Status	Location	Details
app/main.py	COMPLETED	app/main.py	All endpoints, helpers, business rules, and MongoDB aggregations.
REVIEW.md	COMPLETED	REVIEW.md	16 identified starter defects with symptoms, tests, and fixes.
DECISIONS.md	COMPLETED	DECISIONS.md	Five architectural questions answered (150-200 words).
requirements.txt	COMPLETED	requirements.txt	Clean dependencies (fastapi, uvicorn, pymongo, pydantic, dotenv).
README.md	COMPLETED	README.md	Setup, environment configuration, run instructions, and design.
Git Repository	COMPLETED	GitHub (origin main)	Pushed cleanly without .env, secrets, or temporary files.

2. Compliance with Business Rules (R1 - R10)

Rule	Rule Summary	Implementation Details
R1	IST Timezone & Attendance Date	All dates in IST (+05:30). Truncates instants to whole seconds. Overnight shifts (shift_end <= shift_start) assign punch-ins prior to shift_end to previous day.
R2	Late Calculation (10m Grace)	Late only if strictly > 10m (600s) after shift start. Computes floor of whole minutes elapsed from shift_start.
R3	Overtime (>= 30m Threshold)	Floor of minutes between shift_end and punch_out. Counted only if >= 30 minutes, otherwise 0. Handles overnight shift boundaries.
R4	Work Hours (Round Half-Up)	Calculates (punch_out - punch_in) in seconds / 3600, rounded half-up to 2 decimals using decimal.Decimal(ROUND_HALF_UP).
R5	Half Day Detection	Sets half_day = true when rounded work_hours < 4.50. Counts as 0.5 present day.
R6	Presence Statuses	PRESENT, WFH, and ON_DUTY count as present. ABSENT and LEAVE do not.
R7	Working Days	Mon-Fri only. Working days dynamically counted from joined_on for mid-month joiners. Weekend logs add to late/overtime but not present_days.
R8	Rounding Convention	Reported metrics rounded 2 decimals half-up; rates rounded 4 decimals half-up.
R9	Headcount Preservation	Employees with joined_on <= period_end are counted, even if zero logs exist.
R10	Pagination Constraints	1-indexed pagination with max page_size of 100. Total accurately reflects active filters without loading dataset into memory.

3. Summary of 16 Starter Code Defects Fixed (REVIEW.md)

#	Component	Defect Description	Resolution
1	compute_late_minutes	Naive datetimes; computed late > 10 using integer minutes; wrong floor.	Normalized to IST; checked diff_sec > 600; whole minutes floor.
2	compute_work_hours	Banker's rounding in float round(..., 2).	Implemented Decimal ROUND_HALF_UP to 2 decimals.
3	compute_overtime	Did not enforce 30-min minimum; failed overnight shifts.	Enforced >= 30 threshold; added next-day calendar shift for overnight.
4	Employee models	Lacked Pydantic regex/length validations; allowed identical shift times.	Added EMP regex, email validator, and shift_start != shift_end check.
5	POST /employees	Check-then-insert race condition on emp_code.	Created unique index on emp_code; caught DuplicateKeyError (409).
6	created_at field	Used naive local datetime; returned datetime object instead of epoch ms.	Stored UTC BSON date; serialized to epoch ms int in responses.
7	GET /employees	skip = page * page_size skipped page 1; count ignored filter; unbounded size.	Fixed skip=(page-1)*size; filtered count; capped size at 100.
8	Punch-in validation	Accepted non-ms/float timestamps; allowed non-presence statuses.	Strict parse_epoch_ms validation; restricted to PRESENT, WFH, ON_DUTY.
9	POST /punch-in	KeyError 500 when emp_code was unknown.	Looked up employee first and returned 404.
10	Punch-in date	Used local date instead of IST; failed overnight shift day assignment.	Calculated attendance date per R1 considering overnight boundary.
11	Punch-in race	Check-then-insert allowed duplicate punch-ins for same date.	Unique compound index (emp_code, date); 409 on duplicate.
12	ObjectId exposure	Returned internal ObjectId string as 'id'.	Omitted _id and id; serialized strictly according to schema.
13	GET /attendance	Loaded entire 100k collection into memory and sorted in Python.	Implemented streaming cursor sorting (date desc, emp_code asc).
14	GET /attendance	date_from > date_to returned empty list instead of 422.	Validated date ordering and returned 422.
15	Attendance schema	Returned BSON dates instead of epoch ms for punch times.	Serialized punch_in/punch_out to epoch ms integers.
16	GET /health	Returned static 200 without checking database connectivity.	Executed admin ping command; returned 503 if unavailable.

4. Concurrency Handling & Aggregation Architecture (DECISIONS.md)

Database Indexes: Unique indexes on employees.emp_code and attendance_logs.(emp_code, date) guarantee consistency at the engine level. Compound index on (date, late_minutes) accelerates leaderboard filtering, and (date, emp_code) supports fast reverse-chronological paging.

Concurrency & Race Safety: Duplicate punch-ins are serialized by the unique index, ensuring exactly one 201 and 409 for duplicates. Concurrent punch-outs use atomic find_one_and_update({ punch_out: None }) ensuring exactly one 200 and 409 for conflicting requests. Regularization updates employ optimistic concurrency versioning on history length to prevent lost audit trails.

Aggregation Pipelines:
- *Department Summary:* Cross-collection $lookup starting from employees collection preserves zero-log employee headcount.
- *Late Leaderboard:* Uses $setWindowFields with $rank to implement competition ranking (1, 2, 2, 4) with tie preservation.
- *Department Trend:* Uses MongoDB $densify stage to generate contiguous calendar daily series and compute 7-day trailing moving averages in the database.